# Lossy Calibration Archives for Exact Model Repair

Anonymous TACL submission

## Abstract

Low-bit language models use calibration text to choose a small set of allowed weight values. Removing calibration records can change those choices. Recovering the revised model usually requires the original text or a large archive of its internal features. We show how a lossy feature archive can still recover the exact weight choices. The archive stores intervals, and a verifier accepts a choice only when every feature value in those intervals leads to it. Unresolved choices trigger source replay. For a calibration procedure whose feature extractor stays fixed, we prove exact model recovery and equality of the state needed for later deletions. For this target, DistilGPT2 and GPT2 repairs match fresh reconstruction across 42.47 and 84.93 million calibrated weight codes. In a matched study, compression saves 7.01–7.86% of live state, or 1.30–2.24% including the checkpoint, while taking 1.44–2.66 times the optimized lossless request time. The result is a checked storage and source-access tradeoff: exact discrete model decisions can survive the loss of exact feature values.

## 1 Introduction

A language model can be made smaller by replacing each weight with one of a few allowed values. Calibration text helps choose these values by showing which weight errors matter for the model's internal computations. That text is often treated as a one-time input. Yet a calibration collection can later change. A record may be duplicated, corrupted, or removed from a deployment's approved collection. The resulting maintenance task is concrete: rebuild the low-bit weights for the retained records and keep enough state to process the next removal.

A service can retain the text and extract features again, cache exact features, or aggregate their contributions into calibration matrices. Each choice trades storage against later source access. An aggregate alone may not identify the contribution that must be removed.

We study a fourth option: retaining compressed intervals around the feature values. At first this seems incompatible with exact reconstruction. If compression loses a feature value, how can a later computation be exact? The key is that the required output is discrete. Many possible feature values lead to the same rounded weight. The archive need only preserve enough information to distinguish the relevant rounding choices. It need not reconstruct every original feature bit.

Our verifier recomputes retained-set decisions, including changed codes. A numerical solver proposes the calculation; directed bounds check it for every feature matrix described by the archive. When this check cannot resolve a choice, the service can regenerate exact retained-source features.

We study maintenance of a specified calibration procedure. A feature extractor from the original checkpoint is fixed before choosing calibration membership. Each record then has the same features regardless of which other records remain. This *fixed-teacher calibration* supports source-local archives and exact subtraction of source contributions. Ordinary sequential calibration instead allows earlier quantized layers to alter later features. Our target also fixes the ridge, normalization, and checkpoint-only grids. We measure its predictive cost against a matched sequential constructor. The task changes calibration membership; it does not remove information from the checkpoint's pretraining.

We prove exact model and successor-state recovery, and compare complete service transactions across five archive designs.

Adaptive rounding already uses calibration geometry (Frantar et al., 2023; Chee et al., 2023); verified numerics and adaptive predicates already certify discrete decisions (Rump, 2010; Shewchuk, 1997). Our focus is their use when the calibration evidence itself is lossy and the result must support another deletion.

## 2 What must the archive remember?

Consider a service maintaining a four-bit model. Four-bit weights use sixteen allowed values, called *codes*. Calibration adjusts one weight at a time. An earlier rounding error influences the next choice through coefficients computed from the calibration features. Deletion changes those coefficients, which can change later codes. Exact repair therefore requires agreement along this entire adaptive sequence.

**A changed code from uncertain features.** A two-weight example illustrates the information requirement. Let the original weights be $(0.4, 0.4)$, with allowed values $\{-1, 0, 1\}$. Suppose two records have feature vectors $(1, 1)$ and $(-1, 1)$. Their cross-products cancel. The first weight rounds to zero, and the second receives no correction, so the output is $(0, 0)$. Now delete the second record. With a ridge term of one, the coefficient carrying the first rounding error into the second coordinate is

$$c = \frac{x_1 x_2}{1 + x_2^2} = \frac{1}{2}.$$

The second adjusted value becomes $0.4 + c(0.4 - 0) = 0.6$, and its code becomes one.

Suppose the archive retains only $x_1, x_2 \in [0.99, 1.01]$. Direct interval bounds give

$$\frac{0.9801}{2.0201} \le c \le \frac{1.0201}{1.9801}.$$

Every resulting adjusted value lies inside $[0.59, 0.61]$, entirely above the rounding boundary $0.5$. Thus the revised output $(0, 1)$ is exact even though the archive cannot identify either original feature value. This is an algebraic illustration, not an experimental result. If the interval crossed $0.5$, the same check would request more information rather than guess a code.

**Three separate objects.** The model contains deployed codes. The repair state identifies retained records and stores evidence for later deletion. The execution history records work, including replay and failure. Identical codes need not imply identical repair states, and identical states need not imply identical costs. We evaluate these objects separately.

## 3 The reconstruction target

We now define the model that repair must recover. The specification fixes a pretrained checkpoint $W$, a set of identified token records $\mathcal{D}_0$, a teacher $T(W)$, and the tokenizer and record boundaries. It also fixes stage and coordinate order, rounding grids, positive ridge parameters, and the positive original normalization $N_0$. Deleting records changes membership, not these choices. The experiments use the original checkpoint as teacher, $T(W) = W$.

### 3.1 Source-local features

A stage is one calibrated affine transformation. For stage $\ell$ and record $j$, let

$$X_{\ell j} = F_\ell(T(W), j) \in \mathbb{Q}^{d_\ell \times t_j},$$

where $d_\ell$ is the feature width and $t_j$ is the record's token count. $F_\ell$ is a declared finite CPU evaluation. Each successful binary64 output is interpreted exactly as a dyadic rational, an integer times a power of two. This specifies the arithmetic object being reconstructed without replacing the feature extractor by an ideal real-arithmetic network. Unsupported finite operations cause refusal.

The evaluator processes each record independently. For retained membership $R \subseteq \mathcal{D}_0$, concatenate its features in canonical source order to form $X_\ell(R)$. Its regularized second-moment matrix is

$$H_\ell(R) = \lambda_\ell I + \frac{X_\ell(R) X_\ell(R)^\top}{N_0}, \qquad \lambda_\ell > 0. \tag{1}$$

Here $I$ is the identity matrix. The positive ridge makes the matrix positive definite. Fixing $N_0$ keeps the unnormalized ridge $\beta = \lambda_\ell N_0$ constant across deletions. If $T$ tokens remain, multiplying $H$ by $N_0/T$ gives their average Gram plus ridge $\lambda_\ell N_0/T$. Thus the declared policy strengthens regularization relative to the retained-data average as records disappear; changing that policy defines a different target.

### 3.2 Adaptive weight decisions

Suppress stage and output-row indices. Factor the rational matrix as $H = L^\top \operatorname{diag}(\tau) L$, where $L$ is

unit lower triangular and the pivots $\tau_i$ are positive. This reverse LDL factorization determines the coefficient $c_{ih} = L_{ih}$ carrying an earlier rounding error into coordinate $i$. For a base-weight row $w$, define

$$v_i = w_i + \sum_{h<i} c_{ih}(w_h - q_h), \tag{2}$$

$$q_i = \mathrm{nearest}_{G_i}(v_i). \tag{3}$$

This recurrence uses the established quadratic rounding viewpoint (Chee et al., 2023). For a proposed real weight row $y$, the distortion is $(w - y)H(w - y)^\top$. With $y_h = q_h$ for $h < i$, minimize over the remaining real coordinates. The minimizing coordinate $i$ is $v_i$; then round it to $G_i$. The suffix equations in Section 4.2 give this identity. An exact midpoint selects the lower code, and endpoint codes saturate. Collecting all stage outputs defines $Q_{\mathrm{fix}}(R)$; this greedy procedure need not minimize distortion jointly over all discrete codes. The ridge term penalizes weight movement.

For each experimental output row, the grid is $s_{\ell,r}\{-8, \ldots, 7\}$. Its scale is the smallest positive checkpoint-only dyadic scale with at most 24 significant binary digits that covers the base weights and gives finite exactly representable codes and midpoints. The scale is shared across that row's coordinates. Archive precision and weight precision are separate: the archive stores feature evidence, while these sixteen values define the deployed four-bit weights.

Earlier decisions enter later values through (2). A certificate must therefore establish the accepted prefix before using it to certify the next code. The fixed teacher keeps later defining inputs independent of repaired weights.

# 4 Certifying decisions from lossy evidence

Preparation evaluates each source through the frozen teacher and encodes a canonical descriptor $D_{\ell j}$. Decoding returns entrywise bounds satisfying

$$\underline{X}_{\ell j} \le X_{\ell j} \le \overline{X}_{\ell j}. \tag{4}$$

Preparation checks every actual feature value against its bounds. Metadata binds the source and token identities, stage, target, codec, and a commitment to the original feature words.

The codec groups 256 token-major values into blocks. Let $p = 48$ be the index precision and $e_b = \max_{x \in b} \lfloor \log_2 |x| \rfloor$, assigning zero exponent $-1074$. Block $b$ uses step $\Delta_b = 2^{\max(e_b - p + 2, -1074)}$ and signed $p$-bit indices $\lfloor x/\Delta_b \rfloor$. Two-bit tags distinguish closed cells, exact grid values, and exact binary64 escapes near overflow. Endpoints are exactly representable dyadics. Escapes and metadata count toward storage, so 48-bit indices do not imply a uniform 25% archive saving.

## 4.1 From coefficient intervals to codes

A request first validates its predecessor, deletion IDs, retained membership, and target bindings. It keeps each surviving descriptor unchanged and processes stages in order. A numerical solver proposes coefficients. A separate directed verifier bounds their errors over every feature realization in the decoded box. Using the accepted prefix in (2), it obtains an enclosure $[\underline{v}_i, \overline{v}_i]$ for the next adjusted value.

For a grid $g_0 < \cdots < g_{k-1}$, let $m_a = (g_a + g_{a+1})/2$. The decision cell for an interior code $g_a$ is $(m_{a-1}, m_a]$. Endpoint cells extend to infinity. Acceptance requires the entire value enclosure to lie in one cell. The strict lower boundary and inclusive upper boundary implement the lower-code tie rule. A proposal close to a grid value is insufficient if its verified enclosure crosses a cell boundary.

If a box cannot certify the stage within its admitted work, the service may regenerate exact retained features and use a point certificate. It verifies that replayed words and containment match preparation. Access accounting includes every traversed ancestor stage. Temporary exact features do not replace the compressed successor descriptors. Exhausting a CPU, memory, refinement, or replay allowance ends the attempt without committing a complete replacement.

## 4.2 An executable coefficient bound

We give the link between a numerical solve and the adaptive coefficients. For one realized feature matrix $X$, set $K = XX^\top$, $\beta = \lambda N_0 > 0$, and $A = \beta I + K = N_0 H$. Let $S_i = \{i, \ldots, d\}$ be the suffix beginning at coordinate $i$, and let $e_1$ be its first coordinate vector. Then

$$c_{ih} = K_{h,S_i} A_{S_i,S_i}^{-1} e_1, \qquad h < i. \tag{5}$$

To prove the identity, factor $A = L^\top D L$. Scaling $H$ changes the diagonal factor, not $L$. With $S =$

$S_i$, the suffix and cross blocks are

$$A_{S,S} = L_{S,S}^\top D_{S,S} L_{S,S},$$
$$A_{h,S} = L_{S,h}^\top D_{S,S} L_{S,S}.$$

Their product with $A_{S,S}^{-1} e_1$ is $L_{S,h}^\top L_{S,S}^{-\top} e_1 = L_{ih}$: the first column of the unit upper triangular inverse is $e_1$. Since $h < i$, $A_{h,S} = K_{h,S}$, proving (5).

Let $\widehat{z}_i$ be a numerical proposal, held fixed throughout its check. Suppose the verifier establishes $\|e_1 - A_{S,S}\widehat{z}_i\|_2 \le \eta_i$, where $\|\cdot\|_2$ is the Euclidean norm. The coefficient error obeys

$$|c_{ih} - K_{h,S}\widehat{z}_i| \le \eta_i \min\left\{\frac{\|K_{h,S}\|_2}{\beta}, \frac{\sqrt{K_{hh}}}{2\sqrt{\beta}}\right\}. \tag{6}$$

For the first bound, write the error as $K_{h,S}A_{S,S}^{-1}(e_1 - A_{S,S}\widehat{z}_i)$ and use $A_{S,S} \succeq \beta I$. For the second, use $K_{h,S} = X_h X_S^\top$. Each singular value $s$ of $X_S$ contributes $s/(\beta + s^2)$ to the norm of $X_S^\top(\beta I + X_S X_S^\top)^{-1}$. This is at most $1/(2\sqrt{\beta})$, while $\|X_h\|_2 = \sqrt{K_{hh}}$. Multiplication by the residual bound proves the claim.

For uncertain features, directed arithmetic encloses the dot product $K_{h,S}\widehat{z}_i$, the residual norm, the Gram-row norm, and its diagonal uniformly over the feature box. Combining this dot-product enclosure with (6) bounds the coefficient. The argument applies to each realized feature Gram, which is positive semidefinite. It does not assign that property to arbitrary matrices inside an entrywise Gram enclosure. The coefficient intervals then feed the decision-cell check above.

### 4.3 How much uncertainty is tolerable?

The decision check gives a useful criterion for archive precision. Suppose the coefficient intervals for the accepted prefix have centers $\bar{c}_{ih}$ and radii $\delta_{ih} \ge 0$. Define the adjusted center and propagated radius as

$$u_i = w_i + \sum_{h<i} \bar{c}_{ih}(w_h - q_h), \tag{7}$$
$$\rho_i = \sum_{h<i} \delta_{ih}|w_h - q_h|. \tag{8}$$

For every admitted coefficient realization, $|v_i - u_i| \le \rho_i$ by the triangle inequality. If a proposed code has cell $(a_i, b_i]$, the sufficient acceptance conditions are

$$u_i - \rho_i > a_i, \qquad u_i + \rho_i \le b_i. \tag{9}$$

At endpoint codes the corresponding infinite boundary imposes no restriction. These expressions are exact arithmetic identities; an executable evaluation must enclose the centers, products, sums, and comparisons outwardly as well.

**From archive width to coefficient error.** Let $\bar{X}$ be the box center and $E$ its entrywise radius. Set $\epsilon = \|E\|_F$ and $\kappa = \|\bar{X}\|_F$, where $\|\cdot\|_F$ is the square root of the sum of squared entries. Let $\bar{K} = \bar{X}\bar{X}^\top$ and $\bar{A} = \beta I + \bar{K}$. Define

$$\gamma = 2\kappa\epsilon + \epsilon^2, \qquad D_{ih} = \frac{\gamma}{\beta} + \frac{\|\bar{K}_{h,S_i}\|_2 \gamma}{\beta^2}.$$

Every feature matrix in the box satisfies $|c_{ih} - c_{ih}^0| \le D_{ih}$, where $c_{ih}^0$ is the exact coefficient at $\bar{X}$. To see this, write $X = \bar{X} + \Delta$ with $\|\Delta\|_2 \le \epsilon$, giving $\|K - \bar{K}\|_2 \le \gamma$. Both regularized suffix inverses have norm at most $1/\beta$. The resolvent identity bounds their difference by $\gamma/\beta^2$; expanding $K_{h,S}A_{S,S}^{-1} - \bar{K}_{h,S}\bar{A}_{S,S}^{-1}$ proves the bound.

Use $c_{ih}^0$ as the center in (8). Along its accepted prefix, inserting $D_{ih}$ for $\delta_{ih}$ in (8) gives a sufficient archive-width condition via (9). For block cell widths $\Delta_b$, $\epsilon^2$ is the sum of $(\Delta_b/2)^2$ over non-exact entries; exact escapes contribute zero. Smaller cells reduce this bound, but small ridge and large earlier residuals amplify it. This conservative analytical bound is not a prediction of the bounded implementation's success. If two realizations give different codes, no uniform certificate exists. Even when all agree, interval dependence can cause refusal. An exact midpoint has zero two-sided margin; refinement alone need not resolve it.

### 4.4 Choosing the numerical representation

The same coefficient has the token-space form

$$c_{ih} = X_h(\beta I + X_S^\top X_S)^{-1} X_i^\top. \tag{10}$$

This follows by multiplying $X_S^\top(\beta I + X_S X_S^\top) = (\beta I + X_S^\top X_S)X_S^\top$ by the respective inverses. The identities allow a solver to work in either space. For $B_i = \beta I + X_S^\top X_S$ and a fixed proposal $\widehat{t}_i$, a verified bound $\|X_i^\top - B_i\widehat{t}_i\|_2 \le \xi_i$ gives

$$|c_{ih} - X_h\widehat{t}_i| \le \|X_h\|_2 \xi_i/\beta.$$

Indeed, $B_i \succeq \beta I$, so the inverse has norm at most $1/\beta$. Outward enclosures make this bound uniform over a feature box. The token route can propagate it through accumulated weighted features instead of materializing every scalar coefficient.

For width $d$, retained tokens $T$, and output rows $r$, admission uses structural work terms $dT^2$ for token-space coefficients and $d^2T + d^3$ for feature-space coefficients, with $rd^2$ row work for the latter. These are budget proxies, not bounds on the full verification cost. The service admits its arrays and bounded schedule before feature access. Its original dispatcher prefers token space when $4T \leq d$, and streamed feature space otherwise. The final matched study fixes the token route for all point-valued solves. These are declared engineering policies. Actual runtime also depends on interval width, decoding, repeated verification, and replay. Compatible controls receive the same kernels and route choices.

## 5 Correct models and reusable successor states

**Repair procedure.** Prepare each source by evaluating its features, encoding a descriptor, and checking containment. On deletion:
1. Validate the target, predecessor, IDs, and work allowance.
2. Keep surviving descriptors; propose and verify each stage's codes.
3. For an unresolved stage, replay retained sources if permitted, check their words, and certify the point input. Stop only if that stage remains uncertified.
4. Verify all stage extents, serialize state and trust records, and seal completion. Publish a successor only after these checks.

The first guarantee concerns the model produced by a successful request.

**Theorem 1** (Exact code recovery)**.** *Assume that features come from the frozen source-local evaluator and that trusted descriptors satisfy* (4)*. Each accepted certificate must be valid for every feature realization in its supplied box, conditional on its accepted prefix. Any fallback must reproduce the same finite features and certify* (2)–(3)*. If the service commits every required stage, the resulting model is* $Q_{\mathrm{fix}}(R)$*.*

*Proof.* Source locality makes each surviving feature matrix equal to its retained-only reconstruction. It lies inside the prepared box. A valid first-coordinate certificate therefore selects the target code. Suppose the earlier accepted codes equal the target prefix. The next certificate's premise is then satisfied, so its selected code also equals the target. Induction establishes each row. A successful fallback uses the same features and target, giving the same conclusion. The fixed teacher makes this argument apply to every stage without changing later defining inputs. The complete extent check covers the entire committed model. □

The state must also support the next request. Fix the codec, preparer, representation rule, source ordering, provenance policy, and serialization. Let $E(R)$ contain the retained canonical descriptors or the canonical exact pooled Grams required by that representation. Define

$$\mathcal{S}(R) = \mathrm{encode}\big(\text{target}, Q_{\mathrm{fix}}(R), R, E(R), \text{provenance}\big). \tag{11}$$

Provenance here means retained-record identity and target/codec bindings. Execution logs are external. *Trust sidecars* are separate verification records binding exact Gram payloads to source commitments. The canonical state contains the fields in (11); required sidecars are checked separately and included in live bytes. Retrieval tokens or indexes and any source-access obligations are also counted.

**Lemma 1** (Exact source subtraction)**.** *For fixed source-local features,* $K(R) = \sum_{j \in R} X_j X_j^\top$*. Exact subtraction of trusted deleted-source contributions, with original normalization unchanged, equals fresh retained-source accumulation.*

*Proof.* Each source contribution is unchanged by other records' deletion. Finite sums over disjoint source sets are additive in rational arithmetic. Subtracting the removed summands leaves the retained sum. Canonical integer scaling and serialization remove dependence on the order of additions and deletions. □

**Theorem 2** (Canonical successor equality)**.** *Under Theorem 1, suppose descriptors are deterministic and retained unchanged, Gram updates satisfy Lemma 1, and compared executions use the same policies in* (11)*. A successful repair returns* $\mathcal{S}(R)$*. Its bytes equal those of any successful certified fresh reconstruction in that representation. The equality holds after every step of a sequence of successful valid deletions.*

*Proof.* The model agrees by Theorem 1. Retained descriptors agree because their source-local inputs and encoder agree. Pooled Grams agree by exact subtraction and canonicalization. Membership,

target, provenance, and encoding policies are identical, so each serialized component agrees. Apply the same argument at each successor. □

Equality is representation-specific and excludes execution histories. The trusted base includes the finite evaluator, outward arithmetic, runtime and immutable input provenance. Hash commitments detect changed bytes, not semantic containment or a hostile host. Positive ridge ensures invertibility, not completion.

## 6 Evaluation design

We evaluate three questions separately: whether a complete repair is exact, what resources it consumes, and how the fixed-teacher target affects predictions. The service studies use DistilGPT2, with approximately 82 million parameters, and GPT2, with 124 million (Hugging Face, n.d.-a; Hugging Face, n.d.-b). They calibrate 24 and 48 affine stages, respectively. The packed outputs contain 42,467,328 and 84,934,656 codes; unchanged checkpoint components remain necessary for inference. Calibration records are English C4 web text (Raffel et al., 2020), each with 128 tokens.

### 6.1 Comparators and complete accounting

The *cached lossless* arm stores exact feature values. The *modern lossless* arm selects the smallest complete descriptor among raw, zlib6, byte-shuffled zlib6, Zstandard3, and byte-shuffled Zstandard3 encodings, with fixed-order ties. Every candidate encoder is charged. *Cold reconstruction* extracts retained features from source records. The *hybrid exact-Gram* arm stores exact Grams at eighteen width-768 stages and lossless features at six width-3072 stages in DistilGPT2. Our *compressed* arm stores the 48-bit interval descriptors. All arms target the same codes and are compared with a fresh retained-input reconstruction in their respective state representation. The cold arm deliberately writes the lossless successor cache, explaining its identical stored size. It is not a minimal-storage source-only service; such a baseline, retaining only codes, tokens or references, and bindings, is unmeasured. It would require retained sources on every repair. Lossless caches need neither retained nor deleted sources; compression may need retained sources on refusal; the pooled-Gram hybrid may need deleted sources for subtraction.

An oracle traverses retained tokens without reading the candidate predecessor. The audits compare actual model and state payloads; the later studies also compare required trust artifacts. The oracle shares the numerical solver. This checks complete reconstruction and source independence, while leaving implementation errors shared by that solver outside the comparison.

The baseline request clock includes predecessor reads and validation, attempted certificates, accumulation, extraction, fallback, output writes, and cleanup. It excludes the caller's context loading and controller overhead. Optimization and matched tables instead use complete host time, including startup and admission. Comparisons are within each table; absolute times across studies are not paired speed measurements. Nested source extraction is included in its stage clock and is never added a second time.

With preparation $P_a$ and complete cost $C_{a,k}$ on successor transition $k$, representation $a$ has lifetime cost

$$L_a(m) = P_a + \sum_{k=1}^{m} C_{a,k}. \tag{12}$$

This sums successive transitions, not alternative deletions from one root; shared preparation is explicitly attributed.

Live storage includes codes, descriptors or Grams, targets, tokens or retrieval indexes, provenance, and required trust sidecars. We also report a total that adds the common checkpoint. A model embedded in the state is counted once. Runtime and codec installations, and physical audit archives, are separate. Source access counts source–stage record traversals, including ancestors. One record replayed through twenty-four stages costs twenty-four accesses. This counts feature traversal, not distinct disk reads or network requests.

For a service with $n$ independently stored repair states and one shared extra codec installation of $I$ bytes, compression saves total space only if $\sum_{j=1}^{n}(B_{L,j} - B_{C,j}) > I$, where $B_L, B_C$ count each complete state. In Table 1, the modern-lossless differences are 33.959 and 16.997 MB. Charging the entire 23.871 MB codec installation as additional gives 10.089 MB saved at twelve records and 6.873 MB extra at six. Two separate six-record states sharing that installation would save 10.124 MB. This is a sharing calculation, not

a measured deployment. Repair frequency and retention duration determine whether saved byte-time justifies extra compute; no service cost model or deletion frequency was measured.

### 6.2 Four studies and their selection history

The first complete service study uses thirteen previously unexposed training records. It deletes one and then six more, giving $13 \to 12 \to 6$ records while keeping normalization 1664 and ridge $1/100$. Records are sorted by ID; the first is deleted, followed by the next six. It compares all five arms. The second study holds that root and target fixed and evaluates reference and optimized versions of the compressed and modern lossless arms. Both optimized arms receive the compatible token-neighbor kernel. Compression adds a native decoder; modern lossless adds bounded reuse of decoded descriptors. Each package is evaluated as a whole.

The third study matches both checkpoints on four exposed records, deletion $4 \to 3 \to 2$, normalization 512, and ridge $1/100$. Every point-valued solve uses the existing token-space coefficient route, fixed before execution for all four cells and both fresh oracles. Box and Gram policies remain unchanged. Each request runs in a fresh process on the same host with one CPU and 16 GiB memory. The grid-storage guard permits 1,327,104 scalar entries. Sealed preparation artifacts are reused and their costs retained. All timing cells in these studies are single observations. Execution order is fixed, with endpoint reversal in the optimization screen; OS cache is uncontrolled.

The fourth study evaluates the earlier 1664-normalized DistilGPT2 target on untouched articles. It is distinct from the smaller matched targets. Teacher and precision choices preceded it: a nearest-rounded teacher had failed a pooled quality guard with a 5.92% perplexity increase; the original-checkpoint teacher was initially selected using eight exposed articles. A 40-bit archive refused certification, followed by a separately registered 48-bit study. The initial matched GPT2 request also refused after both preparations completed. A separate stage diagnostic motivated the prospective point-token policy above. These choices make the later studies scoped tests of fixed configurations.

## 7 Complete repair results

### 7.1 Storage and source access at a fixed target

All ten arm–endpoint comparisons in the disjoint-root service study match fresh reconstruction for the complete model and successor state. Table 1 shows the storage savings and source access. Compression needs no source traversal on this trajectory; its complete installation cost is treated in Section 6.

The hybrid demonstrates why the access contract matters. At twelve records, it is both smaller and faster than compression, while reading source records. At six, compression is smaller and faster than the hybrid and avoids its 144 accesses. Both lossless caches are faster than compression at both endpoints. The compressed requests use bounded numerical fallback on 68 and 2 rows, respectively, but no source replay. Numerical refinement and source access are therefore distinct costs.

### 7.2 Optimizing both sides of the comparison

All eight requests in Table 2 match fresh model, state, and trust artifacts without source replay. The first optimized endpoint is nearly tied; the second favors modern lossless. Stored sizes remain those in Table 1.

Including attributed shared preparation and both requests, the totals are 10,995.918 seconds for compression and 10,287.226 for modern lossless, a 6.89% observed lifetime penalty. The bundled changes do not isolate a decoder-only effect.

### 7.3 Matched repair across two checkpoint depths

All sixteen complete model, state, and trust comparisons pass for the matched DistilGPT2/GPT2 study. Both audits cover every calibrated stage and code, including the second successor. Table 3 reports all request times. The optimization improves each representation relative to its reference, but optimized compression takes longer than optimized modern lossless at all four checkpoint/endpoint settings.

Storage savings persist across both depths (Table 4). Source avoidance is less uniform. At three GPT2 records, both compressed cells replay 144 source–stage record pairs after refusal at the final-block MLP up-projection. Optimized compressed stage work takes 1,458.767 seconds, including 695.104 seconds of source ex-

| Records | Service arm | Request (s) | Live (MB) | With checkpoint (MB) | Accesses |
|---|---|---|---|---|---|
| 12 | Cached lossless | 4,419.007 | 368.003 | 720.828 | 0 |
| | Modern lossless | 4,579.496 | 367.456 | 720.282 | 0 |
| | Cold reconstruction | 5,973.008 | 368.003 | 720.828 | 288 |
| | Hybrid exact Gram | 4,921.935 | 318.918 | 671.743 | 24 |
| | Compressed | 5,206.569 | 333.497 | 686.322 | 0 |
| 6 | Cached lossless | 922.484 | 195.615 | 548.441 | 0 |
| | Modern lossless | 897.841 | 195.330 | 548.155 | 0 |
| | Cold reconstruction | 1,619.473 | 195.615 | 548.441 | 144 |
| | Hybrid exact Gram | 1,694.311 | 220.247 | 573.072 | 144 |
| | Compressed | 1,351.098 | 178.333 | 531.158 | 0 |

Table 1: Complete DistilGPT2 service on $13 \to 12 \to 6$ records. MB are decimal; live storage includes trust sidecars. Request time excludes caller context loading and controller overhead. Accesses count source–stage traversals. The checkpoint column excludes runtime and codec installation. Each row is one observation; every model/state comparison passes.

| Records | Archive | Ref. (s) | Opt. (s) |
|---|---|---|---|
| 12 | Compressed | 4,444.116 | 3,931.646 |
| | Modern | 3,995.241 | 3,967.523 |
| 6 | Compressed | 1,328.624 | 848.119 |
| | Modern | 939.553 | 752.336 |

Table 2: Complete host times for both optimization packages on the same DistilGPT2 trajectory. Modern denotes the modern lossless cache. Each timing cell is one request.

traction, versus 476.578 seconds for modern lossless. Six of eight compressed requests use no source replay; all eight lossless requests use none. A posthoc comparison of the saved optimized-compressed codes finds 4,150,626 and 3,746,142 predecessor-to-successor changes for DistilGPT2, and 8,993,197 and 8,080,117 for GPT2. Each transition changes 8.82–10.59% of codes. All changes occur in stages certified without replay except 245,692 in the first GPT2 endpoint's replayed stage. This classifies stages, not which individual codes required replay.

At two DistilGPT2 records, compression remains slower without replay: stage work takes 267.560 seconds versus 114.651 for modern lossless. Thus cheaper validation does not remove the certification cost. Both checkpoints share the GPT family; complete code counts measure output coverage, not independent trials.

### 7.4 Predictive cost of the fixed teacher

Quality is evaluated for the earlier DistilGPT2 target with normalization 1664. We select 320 articles without observing losses from 2,778 eligible records in a saved frame of 3,801 complete records. The entire inspected frame, sixty historical exclusions, and the separate calibration frame are protected against reuse. Each article supplies 127 predictions from 128 tokens. Six model roles share these articles: the fixed-teacher and sequential models at each retained endpoint, the original checkpoint, and a nearest-grid control.

The matched sequential constructor uses the same retained records, normalization, ridge, and grids, but obtains later features from its quantized prefix. It isolates the teacher choice within our target family; it is not a comparison with an off-the-shelf GPTQ implementation. A fixed-seed paired article bootstrap uses 20,000 draws shared across endpoints. For article mean negative log losses $\ell_{f,j}$ and $\ell_{s,j}$, the pooled ratio is $\exp(\frac{1}{320}\sum_j(\ell_{f,j} - \ell_{s,j}))$, since articles have equal prediction counts. The maximum is $\max_j \exp(\ell_{f,j} - \ell_{s,j})$. Resampling preserves whole-article pairs; upper bounds use the empirical 97.5th percentile with the next higher order statistic. The frozen guards require each pooled perplexity ratio and its one-sided 97.5% upper bound to be at most 1.05, and each observed maximum article ratio to be at most 1.20. The two upper bounds have nominal familywise level 95%. This is an approximate bootstrap over the admitted frame; the observed maximum describes those articles.

The ratios in Table 5 show a small predictive cost. Candidate perplexities are 60.1053 and 60.3088, compared with sequential values 59.7812 and 59.8807. The paired 95% percentile intervals for the ratios are [1.000455, 1.010342] and [1.002870, 1.011431], both above

| Checkpoint | Records | Ref. comp. | Opt. comp. | Ref. lossless | Opt. lossless | Opt. C/L |
|---|---|---|---|---|---|---|
| DistilGPT2 | 3 | 589.239 | 411.279 | 369.644 | 286.419 | 1.4359 |
| | 2 | 436.691 | 325.131 | 219.802 | 179.706 | 1.8092 |
| GPT2 | 3 | 2,027.290 | 1,544.049 | 733.046 | 580.147 | 2.6615 |
| | 2 | 838.153 | 615.039 | 406.506 | 326.341 | 1.8847 |

Table 3: Complete host times (seconds) for the matched $4 \to 3 \to 2$ point-token study. Lossless denotes the modern cache. C/L divides optimized compressed time by optimized lossless time. Each cell is one observation, with complete equality against its same-checkpoint fresh reconstruction.

| | Live (%) | | Total (%) | |
|---|---|---|---|---|
| Records | 3 | 2 | 3 | 2 |
| DistilGPT2 | 7.78 | 7.01 | 1.84 | 1.30 |
| GPT2 | 7.86 | 7.09 | 2.24 | 1.61 |

Table 4: Matched compressed storage savings relative to modern lossless. Total adds the shared checkpoint to each live-state size; it excludes runtime installs.

| Records | Pooled ratio | Upper bound | Maximum |
|---|---|---|---|
| 12 | 1.005421 | 1.010342 | 1.129140 |
| 6 | 1.007149 | 1.011431 | 1.126961 |

Table 5: Untouched quality: fixed-teacher perplexity divided by matched sequential perplexity on 320 articles. The upper bound is one-sided 97.5%. All prespecified guards pass.

one. Original-checkpoint and nearest-grid control perplexities are 54.9643 and 74.6245. These measurements support the specified tolerance for this target and article frame. The 512-normalized targets in the matched study have no quality evaluation.

All twenty-four CPU/CUDA reference comparisons pass the fixed $10^{-8}$ mean negative-log-likelihood barrier, with maximum difference $1.155 \times 10^{-14}$. The terminal audit checks 1,920 GPU and twenty-four CPU losses, saved sequential codes, original parameters, and recomputed statistics. This evaluates identical saved models across likelihood implementations; calibration itself uses the finite CPU evaluator defined in Section 3.

## 8 Relation to prior work

GPTQ (Frantar et al., 2023) uses asymmetric min–max grids and propagates quantized block outputs; our teacher and grids define a different target. QuIP (Chee et al., 2023) analyzes LDL-based adaptive rounding. Chen et al. (2026) connect reverse-order GPTQ to Babai's nearest-plane algorithm, with a no-clipping distortion bound. These methods assume known calibration geometry. We certify adaptive decisions when only feature enclosures remain. QSS (Tavory et al., 2026) instead uses a frozen schema and additive prediction corrections; deleting schema-training data requires rebuilding its affected instance.

Ginart et al. (2019) combine quantized $k$-means with stability tests and retraining. ExactFun (Xiong et al., 2023) corrects gradients and resumes quantized federated training. MEDU (Lang et al., 2026) compresses distributed-training updates and bounds retraining deviation. These address different learning tasks, whereas our output contract is exact retained-set weight codes and reusable repair state.

QEBVerif (Zhang et al., 2023) bounds quantization-induced inference differences with reachability analysis and MILP. ExecCert (Zhao et al., 2026) uses executable residual certificates and margins for discrete prediction agreement. Our certificate covers an adaptive chain of weight decisions over uncertain features.

## 9 Reproducibility and practical use

**Implementation and data.** The CPU runtime uses CPython 3.12.14, NumPy 2.3.5, OpenBLAS 0.3.30, gmpy2 2.3.2, tokenizers 0.23.2, GCC 12.2, and Zstandard 1.5.7 through python-zstandard 0.25.0 on x86-64 Linux. Binary64 defines the reference target; lower-precision targets need separate correctness and quality evaluation. Quality inference uses an NVIDIA A100 80GB; bootstrap statistics use NumPy 2.2.6. Model citations pin checkpoint revisions. The C4 revision is `1588ec454efa`. The thirteen-record root comes from complete records in the first 2 MiB compressed prefix of English training shard 2. Quality uses validation shard 1 with separate 8 MiB transfer and decoded-frame bounds. Records use the

| Component | Construction, reference and outcome |
|---|---|
| Algebra | 315 matrices: all $2 \times 2$ features in $\{-1, 0, 1\}$ at $\beta \in \{1/4, 1, 4\}$; seeded quarter-dyadic cases with $d \in \{1, 3, 4\}$, $T \leq 4$, entries in $[-3/4, 3/4]$ and $\beta \in \{1/4, 1/2, 5/4\}$. Rational inversion checks 459 coefficients, residual bounds and conditional minimizers. |
| Decision cells | 459 interval/code cases on quarter-grid endpoints in $[-2, 2]$, including midpoint ties and saturation, agree with exact nearest-grid choices. |
| Width bound | 486 boxes: 81 $2 \times 2$ centers, radii $1/16, 1/64$, three ridges above. All 39,366 endpoint/interior realizations pass the rational bound with conservative norm upper bounds. |
| Python helpers | 316 finite sum/product/norm comparisons enclose exact rational values; inputs include cancellation, subnormals and extremes. Another 26 outputs are nonfinite and require refusal. |

Table 6: Generated checks: zero assertion failures. The last row tests final frozen Python helpers against rationals; production C kernels and complete repairs are not exercised.

first 128 tokens without beginning/end markers, with positions reset per source. Selection excludes exact IDs, text/normalized-text/URL hashes, and token prefixes of lengths 16, 32, 64, and 128. Sampling is within these bounded frames, not uniform across C4. Earlier development included WikiText-2 (Merity et al., 2017).

Table 6 summarizes the independent rational validation.

**Executable arithmetic.** Interval products enclose all endpoint products; sums accumulate outward at each step using adjacent binary64 numbers. Exact zero shortcuts are checked explicitly. Squared norm bounds accumulate upper magnitudes. For squared radius $s > 0$, choose even $e$ so $a = s2^{-e} \in [1/2, 2)$. Accept a finite nonnegative proposal $r$ only if $\mathrm{nextdown}(\mathrm{fl}(r^2)) \geq a$; return $r2^{e/2}$. Here fl is binary64 rounding to nearest. Starting from $\mathrm{sqrt}(a)$, test at most eight successive upward-adjacent proposals; $s = 0$ returns zero. Overflow, nonfinite bounds, and failed runtime checks cause refusal. Gradual underflow is required. The native compiler disables fast math and fused contraction. Approximate inverses and BLAS solves propose values; directed checks authorize codes. The service caps point refinement at sixteen coordinates and disables rational fallback: its maximum exact rank and exact-coordinate count are both zero. A midpoint uses the lower code only when its exact value or full enclosure establishes that decision cell.

Feature evaluation retains coordinate-order binary64 products and sums, without BLAS reduction or fused multiply-add. Nonlinear primitives use directed MPFR bounds whose endpoints must round to the same binary64 word; an unresolved primitive refines from 96 to at most 3072 interval bits before refusal. An ordered 104-bit batch path precedes that fallback. Execution binds evaluator sources, Python and loaded math libraries, CPU dispatch fields, primitive backend, and implementation manifests. Proposal libraries are single-threaded; OpenBLAS, OMP, MKL and NumExpr thread counts are one. Changed bindings require requalification, and replay must match the original feature-word commitments, not just versions.

Numerical CPU seconds are 47,767 (service), 35,258 (optimization), 20,526 (matched, including failed/diagnostic work), and 7,526 (quality). The unresolved 12,602-second historical reservation is separate. Archived worker and scheduler clocks overlap; these are not complete all-history totals.

**Inputs and responsibility.** The author-side archive preserves outputs, selections, failures and 1,322 source/protocol copies. Release terms and a fresh external-input reproduction remain pending; no external review supplement is provided. Model cards declare Apache 2.0 for DistilGPT2 and MIT for GPT2; C4's ODC-BY and Common Crawl conditions do not establish rights to every page. Logical calibration deletion leaves external corpora and audit archives intact. No content-level privacy or demographic audit was performed. Generative AI assisted planning, code, analysis, literature checks and writing; the authors remain responsible for all content.

## 10 Conclusion

Lossy calibration evidence can determine exact adaptive weight codes. Uniform certificates, source replay, and canonical state construction make successful repairs reusable for subsequent deletions. The measured operating point trades modest storage savings for additional certification work and workload-dependent source access.

## References


Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. 2023. QuIP: 2-bit quantization of large language models with guarantees. In *Advances in Neural Information Processing Systems*, volume 36, pages 4396–4429.

Jiale Chen, Yalda Shabanzadeh, Elvir Crnčević, Torsten Hoefler, and Dan Alistarh. 2026. The geometry of LLM quantization: GPTQ as Babai's nearest plane algorithm. In *International Conference on Learning Representations*. Author version: arXiv:2507.18553v4.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. ICLR. Primary preprint: arXiv:2210.17323, version 2.

Antonio A. Ginart, Melody Y. Guan, Gregory Valiant, and James Zou. 2019. *Making AI Forget You: Data Deletion in Machine Learning*. Advances in Neural Information Processing Systems 32. Primary proceedings paper.

Hugging Face. n.d.-a. DistilGPT2 model card. Model repository, pinned revision `2290a62682d0`; accessed 28 October 2026.

Hugging Face. n.d.-b. GPT-2 model card. Model repository, pinned revision `607a30d783df`; accessed 28 October 2026.

Natalie Lang, Alon Helvits, and Nir Shlezinger. 2026. Memory-efficient distributed unlearning. *IEEE Access*, 14:24361–24378.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research, 21(140):1–67. Primary journal article.

Siegfried M. Rump. 2010. Verification methods: Rigorous results using floating-point arithmetic. *Acta Numerica*, 19:287–449.

Jonathan Richard Shewchuk. 1997. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3):305–363.

Ami Tavory, Shripad Gade, Tal Sarig, Noam Touitou, and Ido Guy. 2026. *Exact Unlearning via Quantized Sufficient Statistics*. arXiv:2610.07197, version 1.

Zuobin Xiong, Wei Li, Yingshu Li, and Zhipeng Cai. 2023. Exact-Fun: An exact and efficient federated unlearning approach. In *IEEE International Conference on Data Mining*, pages 1439–1444.

Yedi Zhang, Fu Song, and Jun Sun. 2023. QEBVerif: Quantization error bound verification of neural networks. In *Computer Aided Verification*, volume 13965 of LNCS, pages 413–437. Springer.

Ziyu Zhao, Xinyu Wang, Xiao-Wen Chang, and Yixuan He. 2026. *From Mathematical to Executable Certificates for Machine Unlearning*. arXiv:2610.02268, version 1.